AI ENGINEER WORLD'S FAIR · SF · 2026

INNGEST

Your agent's architecture has a HALF-LIFE OF 6 MONTHS

Dan Farrelly
CTO & Co-founder, Inngest

CONTEXT

Who am I?

Dan Farrelly

CTO + Co-founder @ Inngest

- **Built Inngest** - Durable execution platform used by thousands of teams to orchestrate agents, AI workflows, background jobs, and multi-step pipelines.
- **Deep in AI infra daily** - Shipping tools that handle the messy reality of running AI in production (retries, orchestration, state management, observability)
- **Practitioner, not just pundit** - Actually building and operating the systems, not just talking about them

The stack is moving under you

Function calling → Tool use → MCP → CLI → …

LangChain → LangGraph → Deep Agents → …

ReAct → Plan-and-execute → …

Sub-agents → Dynamic Workflows → Loops → …

AGAIN.

INNGEST

THE QUESTION

What part of your architecture survives the next six months?

INNGEST

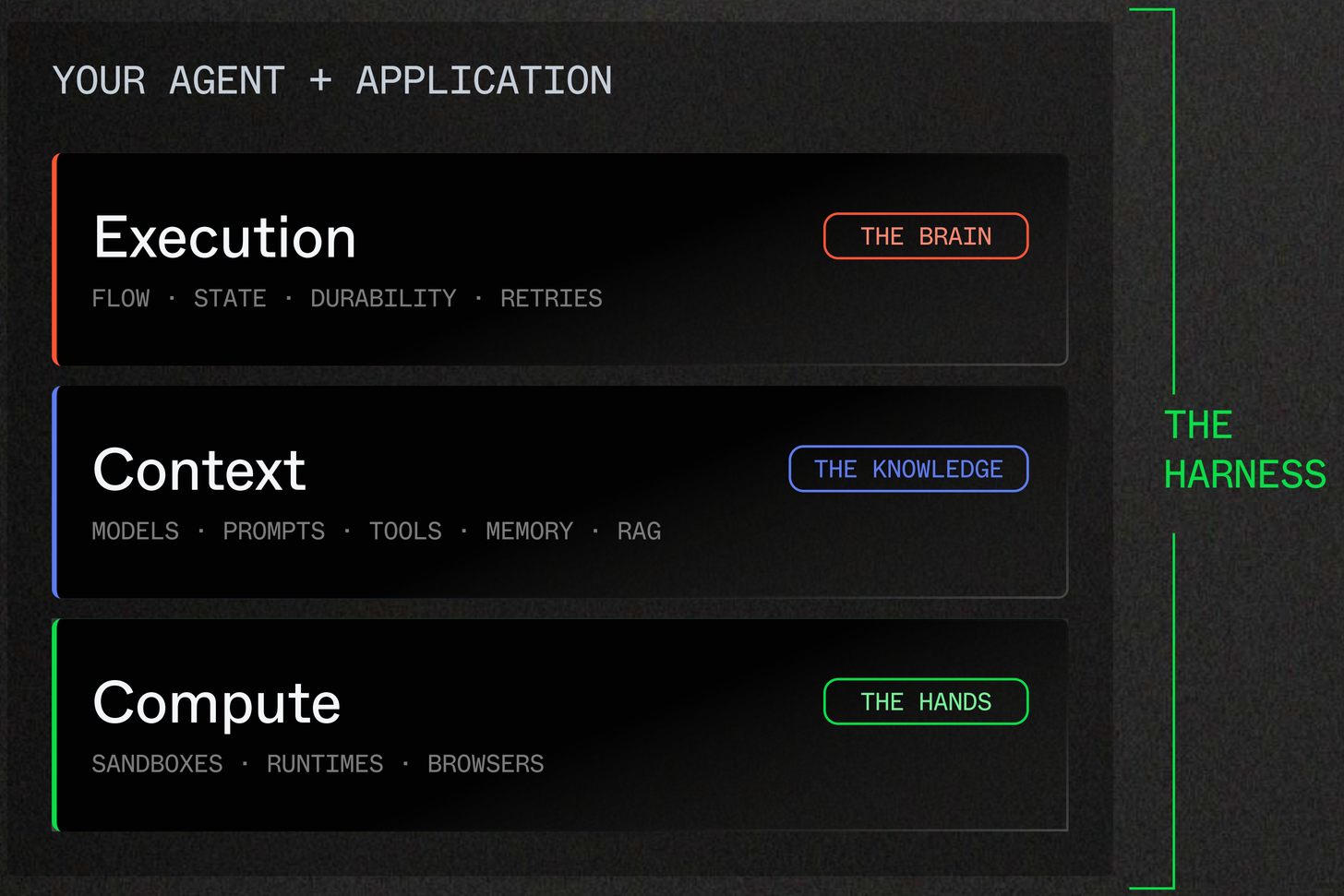
How *did* you architect your agent?

INNGEST

THE HARNESS

Three layers under the agent

Not your product — the infrastructure it sits on.



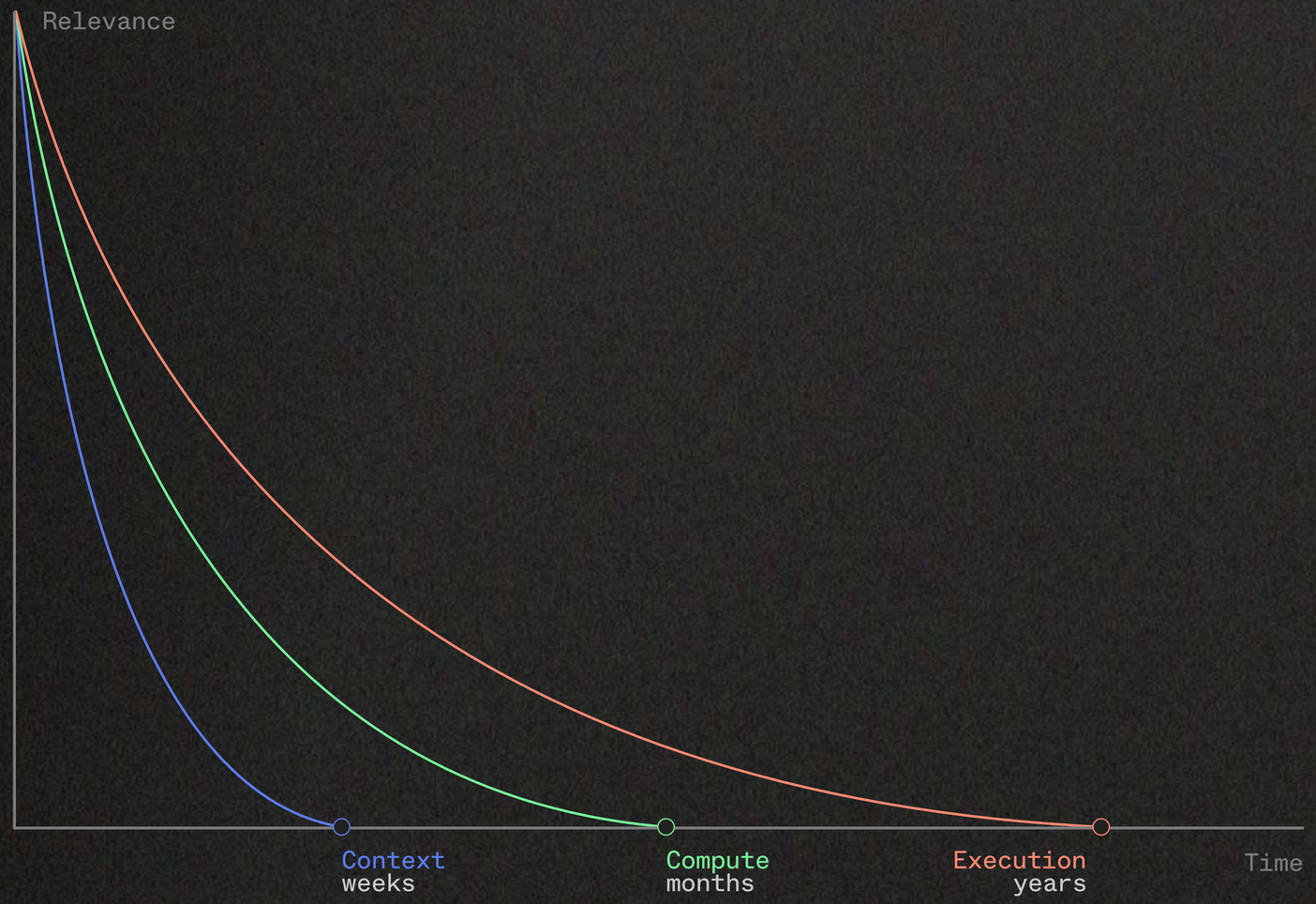
INNGEST

THE THESIS

HALF-LIFE

Every layer decays.
Not at the same rate.

> Decouple the layers.
>
> The part that survives is the part you designed to survive.



INNGEST

DESIGN FOR EVOLUTION

Don't fight it. Embrace it.

Magical frameworks → Very high level abstractions, difficult to adapt

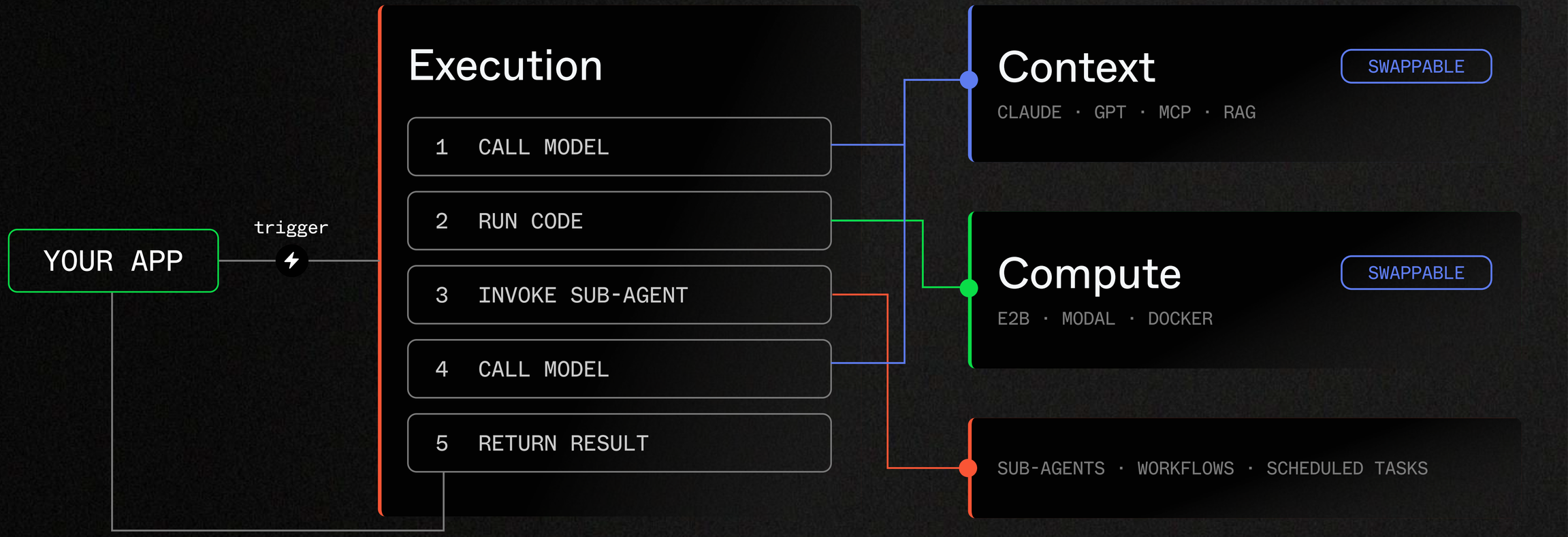
Pre-built harnesses → Complete lack of control

Custom rolled → Complexity grows, abstractions leak across responsibilities

The Execution Layer is the system responsible for running your code reliably — managing how, when, and whether each piece of work completes, independent of the infrastructure it runs on.

THE BRAIN

The harness in motion.



Execution connects the components. Context and Compute are swappable.

INNGEST

ORCHESTRATION — BUILD DURABLE AGENTS

Resume. Don't restart.

An agent 45 minutes in shouldn't lose 45 minutes to a single failure.

Steps 1–37
CACHED · DONE

Step 38
RATE LIMIT

STEP 39
CONTINUE

STEPS

```
const result = await step.run("analyze",
  async () => {
    return llm.chat({
      model: "claude-opus-4",
      messages: [{ role, content }],
    });
  });
```

RETRYABLE · IDEMPOTENT · CACHED

Completed steps are cached — the agent picks up at 38, not step 1.

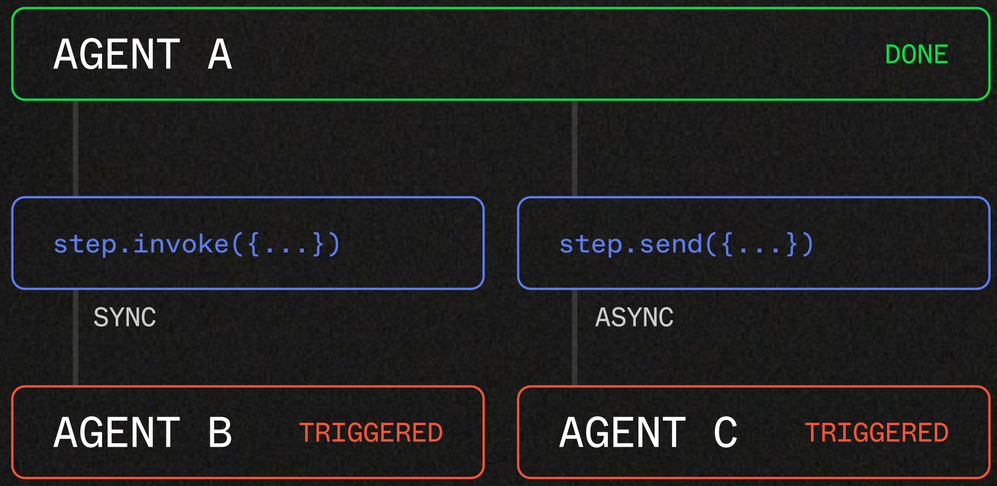
INNGEST

EXECUTION — INVOCATION & COORDINATION

Triggered anywhere. Coordinated async.

FLEXIBLE INVOCATION

SUB-AGENTS: SYNC + ASYNC COORDINATION

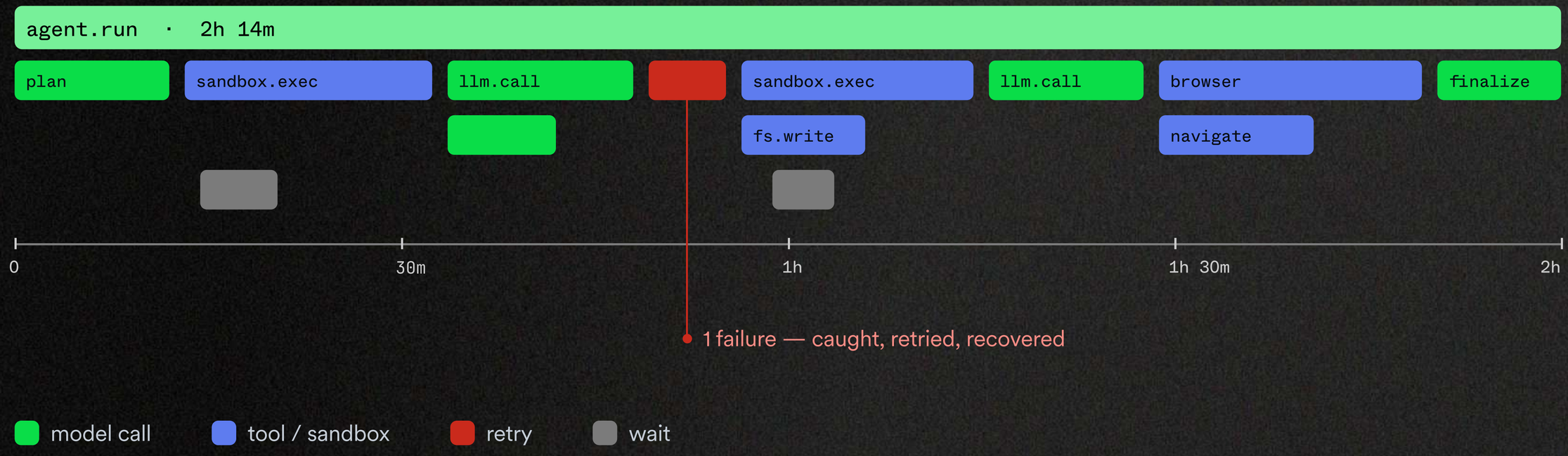


INNGEST

EXECUTION — OBSERVABILITY

See the whole session.

Not one LLM call — the entire run, or session. Models, tools, errors, pauses.



INNGEST

COMPUTE — THE HANDS

A sandbox does work. It doesn't think.

Sandbox

E2B · MODAL · DAYTONA

- STATELESS
- NO MEMORY
- NO RETRIES
- EPHEMERAL
- NO SEQUENCING
- NO RECOVERY

Execution

THE BRAIN

SEQUENCE · STATE · RETRIES · COORDINATION

MANAGES ↓

Compute

THE HANDS

SANDBOXES · RUNTIMES · BROWSERS

INNGEST

What’s next?

WHERE EXECUTION MATTERS MOST

Requirements for emerging systems

What you'll need to evolve for the next six months and beyond

Background agents	Duration: LONG RUNNING · VARIABLE · UNBOUNDED
Dynamic workflows	Triggers: SCHEDULED · ASYNC · DELEGATED
Autonomous loops	Observability: PERSISTENT · INSPECTABLE
Agent factories	Patterns: FAN-OUT/FAN-IN · PLANED · NON-DETERMINISTIC

EXAMPLE

BACKGROUND AGENTS

Minutes to hours. Hundreds of tool calls. And no one watching.

3 HRS
runtime, unattended

200+
tool calls / session

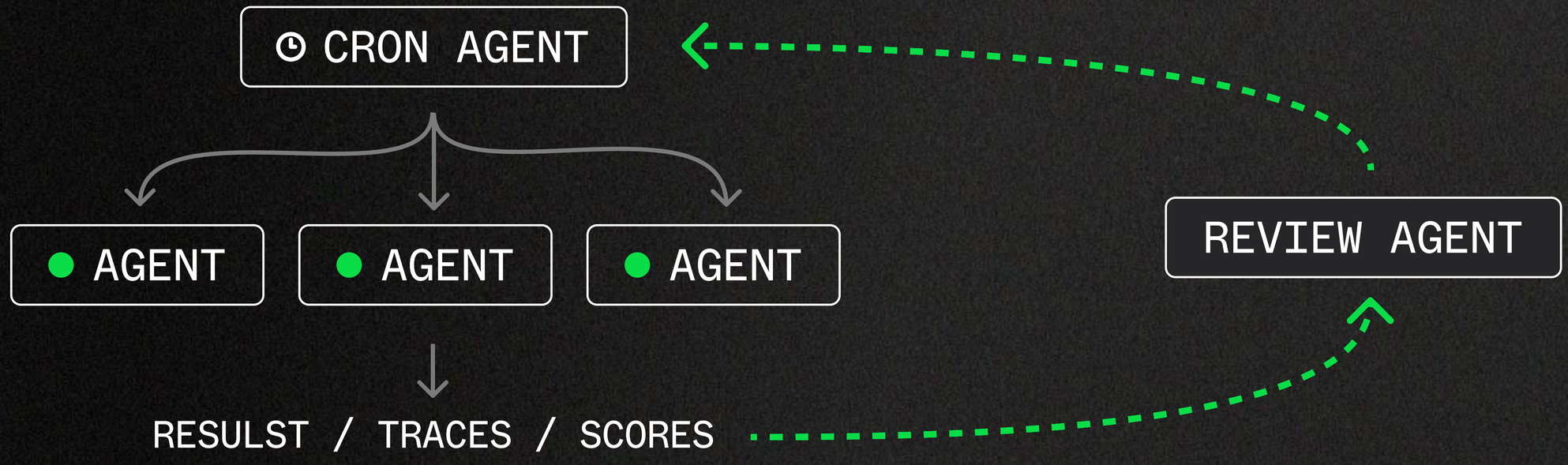
≥ 1
failure, guaranteed

INNGEST

EXAMPLE

LOOP ARCHITECTURES

Crons, scores, iteration



INNGEST

LOOP ARCHITECTURES

THE CRON - THE LOOP ITSELF

```
export const infraHealthCheck = inngest.createFunction(
  { id: "infra-health-check" },
  { cron: "*/30 * * * *" }, // Every 30 minutes
  async ({ step }) => {
    const metrics = await step.run("fetch-service-metrics", async () => {
      return await fetchServiceMetrics(); // error rates, latency, memory, CPU
    });
    const assessment = await step.run("assess-health", async () => {
      return await callLLM(`Given these service metrics, classify overall system health
                  as "normal", "degraded", or "critical". Explain your reasoning.
                  Metrics: ${JSON.stringify(metrics)}`);
    });
    if (assessment.status === "degraded" || assessment.status === "critical") {
      await step.invoke("triage-incident", {
        function: incidentTriage,
        data: { metrics, assessment, services: assessment.affectedServices },
      });
    }
  }
);
```

INNGEST

LOOP ARCHITECTURES

THE TRIAGE AGENT

```
export const incidentTriage = inngest.createFunction(
  { id: "incident-triage", retries: 3 },
  { event: "infra.incident.triage" },
  async ({ event, step }) => {
    const details = await step.run("fetch-detailed-metrics", async () => {
      return await fetchDetailedMetrics({ services: event.data.services });
    });
    const deploys = await step.run("fetch-deploy-history", async () => {
      return await fetchRecentDeploys({ since: hoursAgo(2) });
    });
    let messages = await step.run("correlate-incident", async () => {
      return await callLLM({
        prompt: `Correlate these service metrics with recent deploys.
                 Identify the likely root cause and severity.
                 Metrics: ${JSON.stringify(details)}
                 Recent deploys: ${JSON.stringify(deploys)}`,
        tools: investigationTools,
      });
    });
    while (messages.hasToolCall) {
```

INNGEST

LOOP ARCHITECTURES

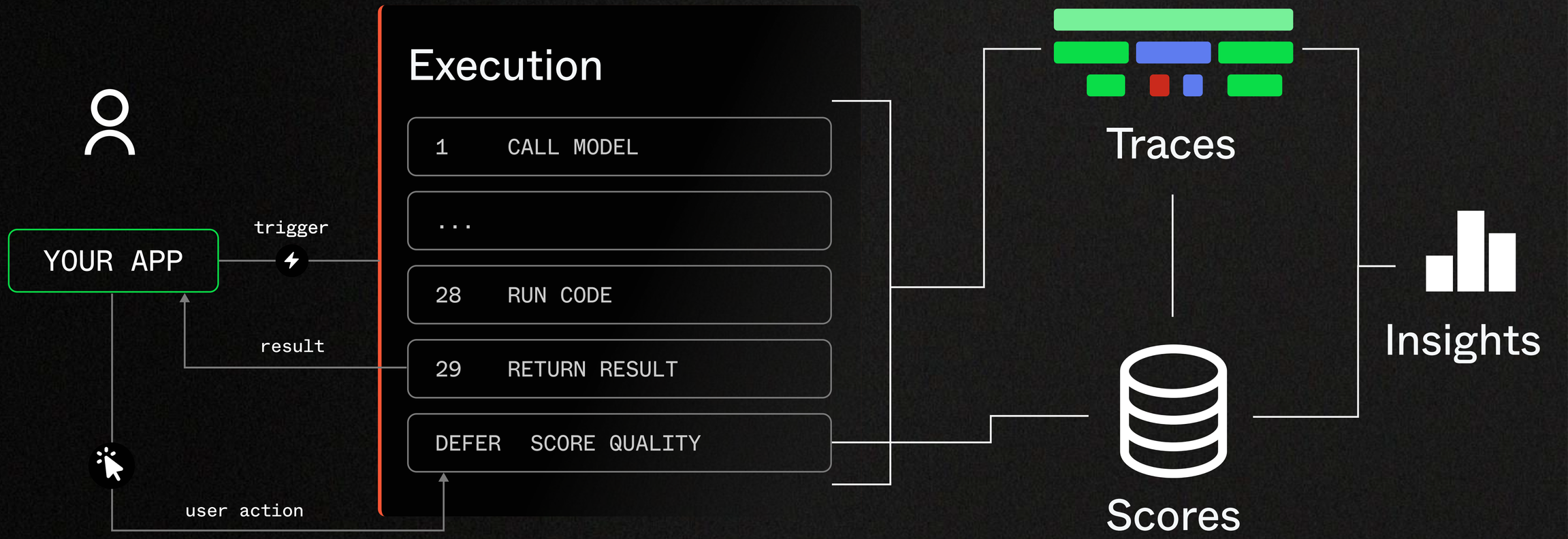
THE TRIAGE AGENT

```
export const reviewSkillPerformance = inngest.createFunction(
  { id: "review-skill-performance" },
  { cron: "0 10 * * 5" }, // Every Friday at 10am
  async ({ step }) => {
    const runs = await step.run("fetch-run-history", async () => {
      return await getInngestRuns({
        functionId: "incident-triage",
        since: daysAgo(7),
      });
    });
    const analysis = await step.run("analyze-performance", async () => {
      const successRate = runs.filter(r => r.status === "completed").length / runs.length;
      const avgDuration = average(runs.map(r => r.duration));
      const incidents = await fetchIncidentOutcomes(); // Did incidents correlate with actual outages?
      return await callLLM({
        prompt: `Review this skill's performance over the past week.
                 Success rate: ${successRate}
                 Avg duration: ${avgDuration}ms
                 Incidents correlated with real outages: ${incidents.confirmed}/${incidents.total}
                 False positives: ${incidents.falsePositives}
```

INNGEST

FEEDBACK LOOPS IN COMPLEX SYSTEMS

Agent observability and evals



Execution layer emits traces and orchestrates feedback based on user action for online evals.

INNGEST

THE EXECUTION LAYER

Inngest

Durable execution for AI agents.

- ✓ Durable steps — retryable, cached
- ✓ Scheduling — cron, delays, debounce
- ✓ Full-session traces — every decision
- ✓ Event triggers — any source
- ✓ Async coordination — agents to agents
- ✓ No infrastructure to manage

Execution

THE BRAIN

FLOW · STATE · DURABILITY · RETRIES

INNGEST

AGENT EVALS

Scoring agent runs

SCORE RUNS ON PRODUCT OUTCOMES

```
const evt = await step.waitForEvent("did-save", {
  event: "agent/report.saved",
  match: "report.id",
});

await inngest.score({
  name: "human_feedback", value: Boolean(evt)
});
await inngest.score({
  name: "accuracy", value: 0.9
});
```

DEFERRED SCORING

```
const result = await step.run("call-llm", () => {
  // ...
});

// Score async after the agent completes:
defer({
  function: tokenEfficiencyScorer,
  data: { result.metadata }
});
```

INNGEST

RECAP

Build your harness.

Execution — STABLE
THE BRAIN

Context — SWAPPABLE
THE KNOWLEDGE

Compute — SWAPPABLE
THE HANDS

Get the execution layer right and iterate quickly.

INNGEST

AI ENGINEER WORLD'S FAIR · SF · 2026

INNGEST

Thanks.

Dan Farrelly
@djfarrelly · inngest.com